

6_sorting.cpp

```
#include <iostream>

using namespace std;

// Function to print array

void printArray(int arr[], int size) {

    for (int i = 0; i < size; i++)

        cout << arr[i] << " ";

    cout << endl;

}

// Bubble Sort

void bubbleSort(int arr[], int size) {

    cout << "\nBUBBLE SORT TRACE\n";

    for (int i = 0; i < size - 1; i++) {

        for (int j = 0; j < size - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {

                int temp = arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1] = temp;

            }

        }

    }

}
```

```
    }  
}  
  
    cout << "Pass " << i + 1 << ": ";  
    printArray(arr, size);  
}  
}
```

// Insertion Sort

```
void insertionSort(int arr[], int size) {
```

```
    for (int i = 1; i < size; i++) {
```

```
        int key = arr[i];
```

```
        int j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j + 1] = arr[j];
```

```
            j--;
```

```
        }
```

```
        arr[j + 1] = key;
```

```
    }
```

```
}
```

```
int main() {

    int original[8] = {64, 34, 25, 12, 22, 11, 90, 50};

    int bubble[8];

    int insertion[8];

    // Copy original array
    for (int i = 0; i < 8; i++) {

        bubble[i] = original[i];

        insertion[i] = original[i];

    }

    cout << "Original Array:\n";

    printArray(original, 8);

    // Bubble Sort

    bubbleSort(bubble, 8);

    cout << "\nFinal Bubble Sorted Array:\n";

    printArray(bubble, 8);

    // Insertion Sort

    insertionSort(insertion, 8);
```

```
cout << "\nInsertion Sorted Array:\n";  
  
printArray(insertion, 8);  
  
return 0;  
}
```

/*

Time Complexity

Bubble Sort

Best Case: $O(n)$

Average Case: $O(n^2)$

Worst Case: $O(n^2)$

Insertion Sort

Best Case: $O(n)$

Average Case: $O(n^2)$

Worst Case: $O(n^2)$

*/

Sample Output

Original Array:

64 34 25 12 22 11 90 50

BUBBLE SORT TRACE

Pass 1: 34 25 12 22 11 64 50 90

Pass 2: 25 12 22 11 34 50 64 90

Pass 3: 12 22 11 25 34 50 64 90

Pass 4: 12 11 22 25 34 50 64 90

Pass 5: 11 12 22 25 34 50 64 90

Pass 6: 11 12 22 25 34 50 64 90

Pass 7: 11 12 22 25 34 50 64 90

Final Bubble Sorted Array:

11 12 22 25 34 50 64 90

Insertion Sorted Array:

11 12 22 25 34 50 64 90

Explanation

1. Bubble Sort

- Compares **adjacent elements**.
- Swaps them if they are in the wrong order.
- After each pass, the **largest element moves to the end**.
- The assignment asks for the array after each pass, so the program prints it after every pass.

2. Insertion Sort

- Starts from the second element.
- Inserts each element into its correct position among the already sorted elements to its left.
- Continues until the whole array is sorted.

Time Complexity

Algorithm	Best	Average	Worst
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$